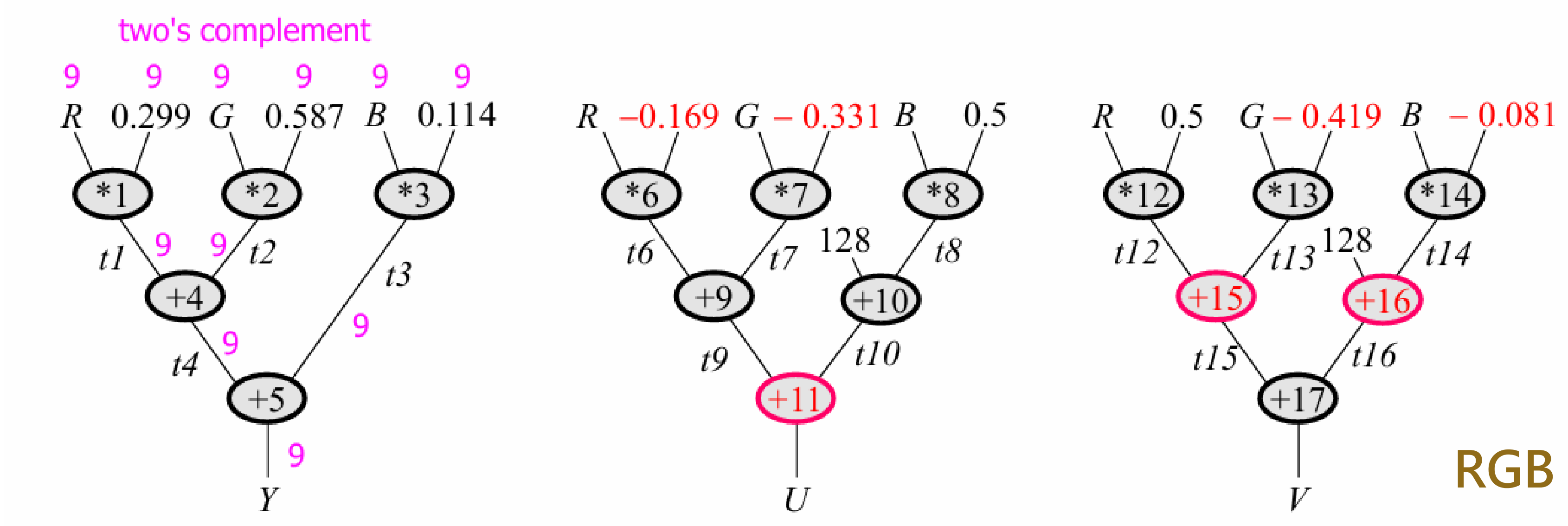
LOGIC SYNTHESIS OF RGB TO YUV

B1130040045 許育菖 2026/06/13

題目說明

- 分別以Verilog製作「 1 multiplier, 1 adder 」以及「3 multiplier, 3 adder 」的 RGB to YUV 邏輯電路，並通過 Synopsys Design Compiler 對電路進行合成(synthesize)，最後比較兩者的輸出結果。



RGB to YUV 公式

實作流程

首先，**1 multiplier, 1 adder**的電路我們在Lab 3已經實作過了。加入教授提供的RGB2YUV_dc.tcl並對輸出檔名做些許修改後，就可以得到輸出結果。

```
#Compile
compile -exact_map
#Report Area
report_area > ./area_report.txt
#Report Timing
report_timing -path full -delay max -max_path 1 -nworst 1 > ./timing_report.txt
#Report Power
report_power > ./power_report.txt
#Save Output File
write -hierarchy -format ddc -output ./RGB2YUV_V1_syn.ddc
write -format verilog -hierarchy -output ./RGB2YUV_V1_syn.v
write_sdf -version 2.1 -context verilog ./RGB2YUV_V1_syn.sdf
write_sdc ./RGB2YUV_V1_syn.sdc
exit
```

排程

接著處理3 multiplier, 3 adder的邏輯電路。

第一步要先進行**排程**，我們可以引用Lab 2得到的結果。

從中發現需要至少5 cycle來完成實現RGB to YUV。

```
[ALU Resource]
- Multiplier: 3
- Adder/Subtractor: 3
  1 pick:     R * 0.299 =   (1)       G * 0.587 =   (2)       R * 0.169 =   (6)
  2 pick:     G * 0.331 =   (7)       B *   0.5 =   (8)       R *   0.5 =  (12)     (1) +   (2) =   (4)
  3 pick:     G * 0.419 =  (13)       B * 0.081 =  (14)       B * 0.114 =   (3)     (6) +   (7) =   (9)     128 +   (8) =  (10)
  4 pick:  (12) -  (13) =  (15)     128 -  (14) =  (16)     (4) +   (3) =     Y
  5 pick:  (10) -   (9) =     U    (15) +  (16) =     V
Total control steps: 5
```

datapath.v

然後透過剛剛得到的排程順序，以version 1的datapath.v為基礎，實作version 2的datapath.v

```
// ------------------------------------------------------------------
// Three multiplier resources.
// ------------------------------------------------------------------
Mul MUL1(.A(mul1_A), .B(mul1_B), .Mul(mul1_out));
Mul MUL2(.A(mul2_A), .B(mul2_B), .Mul(mul2_out));
Mul MUL3(.A(mul3_A), .B(mul3_B), .Mul(mul3_out));

// ------------------------------------------------------------------
// Three adder/subtractor resources.
// ------------------------------------------------------------------
AddSub ALU1(.A(alu1_A), .B(alu1_B), .sub(alu1_sub), .Result(alu1_out));
AddSub ALU2(.A(alu2_A), .B(alu2_B), .sub(alu2_sub), .Result(alu2_out));
AddSub ALU3(.A(alu3_A), .B(alu3_B), .sub(alu3_sub), .Result(alu3_out));
```

```
case (step)
    // Pick 1: t1, t2, t6
    3'd1: begin
        mul1_A = R; mul1_B = C_0299; // t1  = R * 0.299
        mul2_A = G; mul2_B = C_0587; // t2  = G * 0.587
        mul3_A = R; mul3_B = C_0169; // t6  = R * 0.169
    end

    // Pick 2: t7, t8, t12, t4
    3'd2: begin
        mul1_A = G; mul1_B = C_0331; // t7  = G * 0.331
        mul2_A = B; mul2_B = C_0500; // t8  = B * 0.5
        mul3_A = R; mul3_B = C_0500; // t12 = R * 0.5

        alu1_A = t1; alu1_B = t2; alu1_sub = 1'b0; // t4 = t1 + t2
    end

    // Pick 3: t13, t14, t3, t9, t10
    3'd3: begin
        mul1_A = G; mul1_B = C_0419; // t13 = G * 0.419
        mul2_A = B; mul2_B = C_0081; // t14 = B * 0.081
        mul3_A = B; mul3_B = C_0114; // t3  = B * 0.114

        alu1_A = t6;        alu1_B = t7; alu1_sub = 1'b0; // t9  = t6 + t7
        alu2_A = CONST_128; alu2_B = t8; alu2_sub = 1'b0; // t10 = 128 + t8
    end
```

Controller.v

與version 1不同的是，version 2將每個state要做的事交由datapath.v處理。

```verilog
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        Current_State <= S0;
    else
        Current_State <= Next_State;
end

always @(*) begin
    case (Current_State)
        S0: Next_State = start ? S1 : S0;
        S1: Next_State = S2;
        S2: Next_State = S3;
        S3: Next_State = S4;
        S4: Next_State = S5;
        S5: Next_State = S0;
        default: Next_State = S0;
    endcase
end

always @(*) begin
    step = Current_State;
    done = (Current_State == S5);
end
```

實驗結果

實驗結果

Version	Timing	Total Cell Area	Total Area	Cycle
Version 1	9.95 ns	4230.072058	89210.503912	12
Version 2	9.82 ns	9573.580870	175987.769682	6

(包含load)

(詳情見輸出檔案)

實驗心得

這次實驗結合了Lab 2和Lab 3，但我一開始沒注意到需要使用Lab 2的排程結果，導致浪費了一大堆時間。我認為這學期的實驗最有挑戰性的地方在於幾乎每個實驗都跟前一個實驗有關聯，如果前一個實驗沒有做出來，那下一個就要做的很辛苦。

我還滿慶幸我前幾個實驗都沒有偷懶，不然無法想像實作Lab 4的時候要重新排程和設計電路要花多少時間。一開始在實驗課時，其實我撰寫的程式碼有錯誤，導致version 2的輸出area大於version 1，後來發現是由於我在datapath沒有正確使用到多個multiplier和adder的關係。感謝助教當時讓我可以回去慢慢想，不然我當天沒睡好，基本上是想不出來的~